

Sprint 1 — Production Pipeline Integrity

Branch: `feat/generation-quality-engine`

Scope: The live generation pipeline only — `Requirement` → `Classification` → `Scenario Planner` → `LLM` → `resolveType()` → `Quality Validator` → `Output`.

Explicitly out of scope this sprint: Requirement Understanding, Validation Planning, the LLM prompt, and any new intelligence engine. This was a disciplined, single-focus fix of the production defect, not a capability expansion.

The bug we were actually chasing

The customer-visible symptom was a suite that reported “**11 Positive · 1 Negative · 0 Edge**” even when the planner had deliberately created negative and edge scenarios. Coverage the planner asked for was disappearing by the time the suite was graded and shown.

Root cause (confirmed in code, not guessed): `resolveType()` in `test-coverage-engine.ts` ended with a `default: return POSITIVE`. Whenever a generated case could not be tied back to its planned scenario — no `coverageType` on the case, and a `scenarioIndex` that was null or out of range — the function silently labelled it **Positive**. Every leak in the planner→LLM handoff therefore drained into the Positive bucket. The number looked plausible, so nobody saw the loss. A second, quieter version of the same default lived in `analyzeCoverageMix ( tc.coverageType || 'positive' )`.

The principle we adopted: **an unclassifiable case is a pipeline defect, not a Positive**. Silence is the bug.

What changed — the four phases

Phase 1 — Instrument the live path

Every generation now logs a side-by-side trace so the collapse is observable instead of hidden:

- **Planner → resolved trace** — for every returned case: `scenarioId / via / resolved type / title`, alongside the full planner list (`id / coverageType`). The `via` field states exactly how each case was classified (`case`, `scenarioId`, `scenarioIndex`, or `unresolved`).
- **Coverage Loss metric (permanent)** — one line, every run:
`Planner created: N → LLM returned: M → Unknown: K → Coverage Loss: X%`.
 This immediately says whether information leaked and where. It is now both a log line and a structured field (`qualityReport.coverageLoss`) so History/UI can surface it without re-auditing.
- When `K > 0`, a `logger.error` fires listing the offending cases — a leak is now loud.

Phase 2 — Fix the real problem (no silent Positive, ever)

`resolveType()` no longer has a Positive fallback. Resolution is now an explicit, ordered lookup:

1. the case's own `coverageType` (via `case`), else
2. the case's `scenarioId` matched against the planner map (via `scenarioId`), else
3. the case's `scenarioIndex` into the planned scenarios (via `scenarioIndex`), else

4. the explicit `unknown` sentinel (via `unresolved`).

`unknown` → **logged as an error, counted, and it fails the gate**. It is never regenerated into a fake Positive and never silently absorbed. Unknown is treated as the defect it is, so a developer investigates the handoff rather than trusting a wrong green number.

Phase 3 — Stable IDs: the planner owns coverage, the LLM owns wording

Coverage type is now decided **once**, by the planner, and carried by a stable `scenarioId` . To make that possible on the live path, `generateTestCoverage` now threads the planner's `plannedScenarios` (`{ id, coverageType }`) out of the function, and `resolveType` classifies purely by looking a case's `scenarioId` up in that map. **No re-classification, no keyword-matching, no inference happens downstream**. The LLM's job is narrative (title, steps, objective); the planner's classification is merged in as the source of truth.

Phase 4 — The Quality Gate is a real gate

The auditor now produces an explicit `gateReasons[]` and a `passed` verdict. A suite **fails** when any of these hold:

- one or more cases are `unknown` (coverage loss),
- coverage risk is HIGH,
- a selected family came back empty / below its minimum share,
- near-duplicate clusters were detected.

At the API (`routes/test-coverage.ts`), when the gate fails the pipeline **does not persist and does not return a success** — it responds `422` with the reasons, the Coverage Loss metric, and the unsaved suite so a developer can inspect exactly what was produced. “No export. No save. No response.”

Benchmark: where is the 11 / 1 / 0 born? (evidence, not assumption)

Before merging, one benchmark was run to prove where the imbalance originates, rather than assume it. The real production `planScenarios()` was executed for the “Add Employee” requirement (`scripts/coverage-origin-benchmark.ts`), capturing the coverage distribution at every checkpoint we can run deterministically here.

Selection	Planner Positive	Planner Negative	Planner Edge	Total
positive + negative + edge (Standard)	13	7	2	22
positive + negative + edge (Deep)	16	7	3	26
positive only	13	0	0	13

The planner does NOT produce 11 / 1 / 0. With negative and edge selected it emits a balanced suite (7 negatives, 2–3 edges); with positive-only it correctly emits 13 / 0 / 0. There is no selection under which it produces the customer’s “1 Negative / 0 Edge” shape — the planner emits either ~7 negatives or 0, never 1.

Conclusion (Case 1, not Case 2): the imbalance is NOT born in the planner. A final suite of 11/1/0 (12 cases) derived from a balanced 22-case plan — with all-but-one survivor landing in Positive — is the exact signature of the old `resolveType()` silent default: metadata lost → defaulted Positive, one case kept its type → stayed Negative, edges dropped. This is precisely what Sprint 1 fixes.

Where the planner IS worth a later look: it is positive-heavy (13 of 22 ≈ 59%, roughly one Positive per acceptance criterion). That is a coverage-**balance** concern for Sprint 2/3 (the coverage ceiling) — NOT the 11/1/0 collapse. The planner still generates negatives and edges; it does not zero them out.

Honest limits of this benchmark:

- No live LLM/DB in this environment, so checkpoint 2 (LLM output) is inferred, not observed. The planner numbers are the deterministic ceiling; the new trace logs (`Coverage classification trace` + `Coverage Loss metric`) will print the exact planner→LLM→final numbers on one run in the running app — the final proof.
- The benchmark uses the repo’s gold “Create Employee” requirement; the customer’s literal requirement text may differ. Re-run the same script (or read the live trace) against their exact input to confirm for that case.
- If the customer had selected positive-only, 13/0/0 is correct behaviour, not a bug — worth confirming what they actually selected.

Honest verification status

What I **did** verify here:

- `npx tsc --noEmit` — clean.
- `npx jest tests/unit/generation-quality-engine.test.ts` — **25/25 pass**, including 4 new tests that prove an `unknown` case fails the gate, is counted, is never called Positive, and that the Coverage Loss metric is present/accurate (`25%` on a 1-of-4 loss) and absent when no planner count is supplied.

- `scenario-planner.test.ts` , `scenario-builder.test.ts` — pass.
- The pre-existing failing suites (`architecture-contract` , `intelligence-orchestrator` , `intelligence` , `profile-diff-engine` , plus two self-executing script tests that call `process.exit`) fail **identically on a clean tree** — confirmed by stashing my changes. They are **not** caused by this work and were **not** touched.

What I **did not** verify here — stated plainly:

- I could not run the full app + database + real LLM + frontend in this environment, so the end-to-end 422 blocking behaviour has **not** been exercised against a live request. It is implemented and type-checks, but “the API blocks a failing suite in production” is **not** independently confirmed by me.
- Because a hard block changes the behaviour of a running product, the Phase-4 enforcement ships **behind a flag** (`GEN_QUALITY_GATE_BLOCKING` , default `false`). You asked for the gate ON; the tradeoff is that turning it on will start rejecting suites that previously shipped. I recommend enabling it in staging first, watching the Coverage Loss metric and `gateReasons` in the logs, then flipping it on in production. Everything else in Sprints 1’s fix (no-silent-Positive, instrumentation, stable IDs, the metric) is **always on**.

Flags (documented in `.env.example`)

Flag	Default	Effect
<code>GEN_QUALITY_ENGINE</code>	<code>true</code>	Compute the quality report + Coverage Loss metric.
<code>GEN_QUALITY_REGEN</code>	<code>false</code>	Opt-in one-shot regeneration of empty selected families.
<code>GEN_QUALITY_GATE_BLOCKING</code>	<code>false</code>	Phase 4 hard block: fail → 422, no persist, no export.

Files touched

- `src/engines/test-coverage-engine.ts` — Phase 1 instrumentation, Phase 2 `resolveType` rewrite (no Positive fallback), Phase 3 planner-scenario threading + `scenarioId` resolution, `plannerScenarioCount` wired into the auditor.
- `src/engines/generation-quality-engine.ts` — `unknownCount` , `coverageLoss` , and `gateReasons[]` added to `QualityReport` ; `passed` now derives from `gateReasons` .
- `src/api/routes/test-coverage.ts` — Phase 4 flag-gated hard block before persistence.
- `tests/unit/generation-quality-engine.test.ts` — 4 new regression tests.
- `.env.example` — the three quality-gate flags documented.

Deliberately deferred (not built this sprint)

- **Sprint 2** — audit of the Requirement Understanding and Validation Planning engines (YES/PARTIAL/NO capability matrix; reuse / extend / replace decisions).
- **Sprint 3** — Validation Intent.
- **Sprint 4** — Planner → LLM Writer contract.

No new intelligence engine was built. The production pipeline was fixed first.